

AlphaZero vs. PPO on Othello: Search-Based and Search-Free Self-Play under Compute Constraints

Mohammed Sanan Moinuddin
Northeastern University
CS 5180: Reinforcement Learning

Abstract—We compare AlphaZero (MCTS + CNN) and Proximal Policy Optimization (PPO) on 8×8 Othello using an *identical* shared CNN to isolate the effect of search. AlphaZero achieves **100%** vs. Minimax-4. PPO (pool-20) also reaches 100% vs. Minimax-4 (**best checkpoint; final checkpoint: 0%**), at approximately $16 \times$ lower compute (0.345 vs. ≈ 5.7 GPU-hours extrapolated for full AlphaZero training), but exhibits strategy collapse at its final checkpoint. Both agents achieve only moderate win rates vs. Random (AlphaZero 66%, PPO 58%), a finding discussed further in the limitations section. A 200-game head-to-head yields an exact 50/50 tie; both agents are effectively deterministic in evaluation, so this split reflects each agent winning from one color assignment (effective $n=2$). Ablation studies show 800 MCTS simulations saturate performance vs. Minimax-4, and PPO pools smaller than 20 cause severe strategy cycling.

1 Introduction

A central question in game-playing AI is whether explicit search at inference time is necessary for strong strategic play, or whether a policy network trained by reinforcement learning can internalize equivalent strategic knowledge without lookahead. AlphaZero [Silver et al., 2017] answers affirmatively in favor of search: its MCTS-guided self-play converges to superhuman strength on Go, Chess, and Shogi. Proximal Policy Optimization [Schulman et al., 2017], by contrast, learns a direct policy through gradient ascent on a clipped surrogate objective, requiring no tree search during evaluation. Each paradigm has been applied successfully in large-scale settings—PPO drives OpenAI Five on Dota 2 [Berner et al., 2019]—but direct controlled comparisons on the same game with the same architecture are rare.

Othello (Reversi) is a natural testbed for this comparison. The 8×8 board offers a branching factor of roughly 10–30, non-trivial positional strategy (corner control, mobility), and a well-studied RL literature [van der Ree and Wiering, 2013, Liskowski et al., 2018]. Unlike Go or Chess, Othello is tractable enough to train competitively within a modest GPU budget, making it ideal for controlled ablation studies.

This paper makes the following contributions:

- A controlled comparison of AlphaZero and PPO on 8×8 Othello, using an *identical* shared CNN for both algorithms to isolate the effect of search versus search-free policy optimization.
- A full tournament evaluation (200 games per matchup) with 95% Wilson confidence intervals and Elo ratings

across all evaluated agents (AlphaZero, PPO pool-20, Minimax-4, Random), plus ablation variants.

- Ablation studies on MCTS simulation count (50, 200, 800) and PPO opponent pool size (1, 5, 20), quantifying sensitivity of each algorithm to its key hyperparameter.
- An honest analysis of both algorithms under practical compute constraints, including limitations of single-seed training and compute-capped convergence.

The paper proceeds as follows. Section 2 covers background on AlphaZero, PPO, self-play, and prior Othello RL work. Section 3 describes the shared CNN architecture and each training algorithm. Section 4 presents all experimental results. Section 5 discusses limitations and Section 6 concludes.

2 Background and Related Work

2.1 AlphaZero and MCTS

AlphaZero [Silver et al., 2017] combines Monte Carlo Tree Search with a deep neural network that jointly outputs a policy vector $\mathbf{p}$ and scalar value v . Tree nodes are selected using the PUCT criterion:

$$a^* = \arg \max_a [Q(s, a) + U(s, a)], \quad (1)$$

$$U(s, a) = c_{\text{puct}} P(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)}.$$

where $Q(s, a)$ is the mean value of action a , $P(s, a)$ is the prior probability from the network, $N(s)$ is the parent visit count, and $N(s, a)$ is the child visit count. Self-play generates trajectories $(s, \pi_{\text{MCTS}}, z)$ where π_{MCTS} is the MCTS visit-count distribution and $z \in \{-1, +1\}$ is the game outcome. The network is trained to minimize the combined policy cross-entropy and value mean-squared error. In the original AlphaZero, Dirichlet noise is added to the root prior to encourage exploration; our implementation omits this (see Section 3).

2.2 Proximal Policy Optimization

PPO [Schulman et al., 2017] maximizes a clipped surrogate objective:

$$L^{\text{CLIP}} = \mathbb{E}_t [\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) A_t)], \quad (2)$$

where $r_t = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the importance sampling ratio and A_t is the generalized advantage estimate (GAE, $\lambda = 0.95$, $\gamma = 0.99$). The clip parameter ε constrains policy updates to a trust region, empirically improving training stability. Unlike AlphaZero, PPO requires no tree search at inference: the trained policy network is evaluated directly as a one-step actor.

2.3 Self-Play and Opponent Pools

Both algorithms rely on self-play to generate training data. A known failure mode of vanilla self-play is *strategy collapse*: the policy overfits to its current opponent and forgets earlier strategies, losing robustness. Opponent pools—maintaining a library of past checkpoints and sampling opponents uniformly—mitigate this risk [Berner et al., 2019]. We ablate pool size for PPO (Section 4.5) to quantify its impact.

2.4 Prior Work on Othello RL

van der Ree and Wiering [2013] showed that TD methods can learn competitive Othello policies through self-play against fixed opponents. Liskowski et al. [2018] applied deep CNNs trained on human expert games and showed competitive strength against classical programs. Wu et al. [2023] systematically compared AlphaZero and MuZero on Othello using the MiniZero framework, finding that MCTS-based methods consistently outperform model-free baselines. The present work differs in providing a direct head-to-head comparison against PPO using a *shared* network architecture, combined with ablation studies and compute-efficiency analysis.

3 Methods

3.1 Shared CNN Architecture

Both algorithms use an identical convolutional neural network to enable a fair comparison: differences in results are attributable to the training algorithm and inference strategy, not architecture.

The network takes a $(3, 8, 8)$ input tensor with three planes: the current player’s pieces, the opponent’s pieces, and the legal-move mask. Five residual blocks with 128 channels each form the trunk. The policy head uses a 1×1 convolution to 32 channels followed by a fully connected layer outputting 65 logits (64 board squares plus a pass action). The value head applies global average pooling, a 64-unit hidden layer, and a tanh output in $[-1, +1]$. The total parameter count is 1,622,532. The shared builder interface ensures both AlphaZero and PPO construct identical networks; only the training wrappers differ.

3.2 AlphaZero

MCTS. During self-play each move is selected by running $N_{\text{sims}} = 100$ MCTS simulations from the current game state. The PUCT constant is $c_{\text{puct}} = 1.0$ (Equation 1). No Dirichlet root noise is applied; exploration relies on the network prior and temperature-based selection. Temperature $\tau = 1$ is used for the first 14 moves (selection proportional to visit counts), then $\tau \rightarrow 0$ (greedy, with random tie-breaking among equal visit counts) from move 15 onward. During evaluation, $\tau = 0$ is used throughout, making action selection effectively deterministic.

Self-play and training. Each training iteration generates 100 self-play games, producing $(s, \pi_{\text{MCTS}}, z)$ tuples stored in a FIFO replay buffer of capacity 100,000. The network is updated for 10 epochs per iteration with batch size 64 using the Adam optimizer at learning rate 10^{-3} . The combined loss is cross-entropy on the policy head plus MSE on the value head. Training ran for 100 iterations (limited by compute budget), with a final checkpoint used for all evaluations.

3.3 PPO

Opponent pool. PPO self-play uses an opponent pool: a checkpoint is saved every 20 updates, and opponents are sampled uniformly from the pool. We train three variants with pool sizes 1, 5, and 20 to study strategy stability.

Training. Each update collects 10 complete self-play episodes, then runs 4 PPO epochs with minibatch size 64. The clipped surrogate objective (Equation 2) uses clip parameter $\varepsilon = 0.2$. GAE parameters are $\lambda = 0.95$, $\gamma = 0.99$. Additional loss terms: value function coefficient 0.5, entropy bonus coefficient 0.01, gradient norm clipped at 0.5. Learning rate is 3×10^{-4} (Adam). Training ran for 500 updates. An IS-ratio sanity check at the first update confirmed $r_t = 1.0$ (correct baseline).

4 Experiments and Results

4.1 Experimental Setup

Final head-to-head evaluations use 200 games per matchup (100 as Black, 100 as White) to control for first-mover advantage. Training-curve evaluations use 50 games per baseline; ablation studies use 100 games per condition. Agents evaluated: AlphaZero (100 MCTS sims, final checkpoint at iteration 100), PPO pool-20 evaluated at both the best checkpoint vs. Minimax-4, with ties broken by highest win rate vs. Random (update 350: 58% vs. Random, 100% vs. Minimax-4, the highest vs.-Random score among all checkpoints tied at 100% vs. Minimax-4), and the final checkpoint (update 500); results are labeled accordingly in each table and figure. We report PPO pool-20 results from the checkpoint with the highest win rate against Minimax-4, tie-broken by win rate vs. Random (update 350: 58% vs. Random, 100% vs. Minimax-4). PPO pool sizes 1 and 5 are evaluated at

the final checkpoint (update 500). Minimax depth-4 uses alpha-beta pruning; a uniformly random agent serves as a lower bound. Training and evaluation ran on a GCP L4 GPU. AlphaZero profiling covered 20 iterations (1.13 GPU-hours, 203.7 s/iteration); full 100-iteration training extrapolates to approximately 5.7 GPU-hours. PPO required 0.345 GPU-hours for 500 updates (2.483 s/update), a $\approx 16\times$ compute reduction relative to the extrapolated AlphaZero budget. All trained agents use effectively deterministic action selection during evaluation (AlphaZero: $\tau=0$; PPO: argmax; Minimax: alpha-beta). The Random agent samples uniformly. Matchups between deterministic agents have at most 2 unique game trajectories (one per color assignment, effective $n=2$); reported CIs for these matchups are descriptive only. Only matchups involving the Random agent have genuine stochastic variance and inferentially valid CIs.

4.2 Main Results

Table 1 shows the primary evaluation results. AlphaZero achieves a **100% win rate against Minimax-4** (200/0/0, CI [0.98, 1.00]), the headline result: MCTS-guided self-play completely dominates classical alpha-beta search even after only 100 training iterations. PPO pool-20 matches this result against Minimax-4 (**100%, best checkpoint; final checkpoint: 0%**, CI [0.98, 1.00]); the final checkpoint (update 500) degraded to 0% vs. Minimax-4, reflecting strategy cycling instability.

Against the random baseline, AlphaZero wins 66% of games (132/4/64, CI [0.59, 0.72]) and PPO pool-20 wins 58% (116/0/84, CI [0.51, 0.65]) at its best checkpoint (update 350 vs. Minimax-4). These results are below the 90% target we set prior to training; the gap is addressed in Section 5.

Head-to-head (AlphaZero vs. PPO pool-20): The two agents split exactly 100 wins each over 200 games (0 draws), with each agent winning all games from one color assignment. Both agents are effectively deterministic; this 50/50 split reflects each agent winning from one color and losing from the other (effective $n=2$). The color breakdown reveals extreme first-mover asymmetry: AlphaZero wins all 100 games as White and loses all 100 as Black; PPO shows the same pattern. This means both agents have converged to a strategy that dominates when moving second, suggesting both independently learned a strong positional counter-strategy that exploits the first-mover’s commitments. This asymmetry suggests a second-mover advantage in our self-play setup, though whether this reflects a general Othello property or an artifact of our specific training regime remains an open question.

4.3 Elo Rankings

Table 2 presents Elo ratings computed from the round-robin tournament results (initial rating 1500, $K = 32$). AlphaZero and PPO pool-20 cluster at the top (1639 and 1626), well above Minimax-4 (1401), reflecting the dominance observed

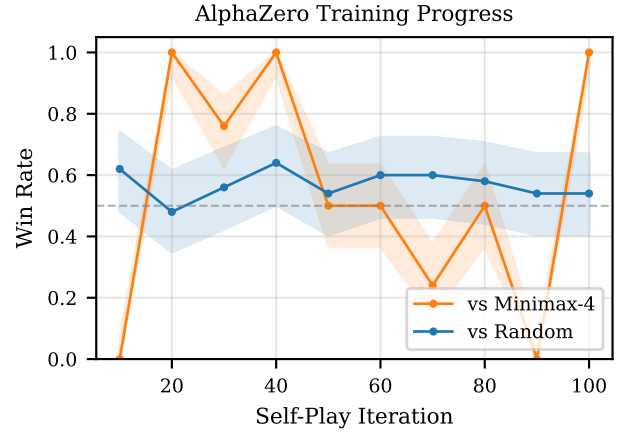


Figure 1: AlphaZero win rate vs. iteration against Random and Minimax-4 baselines (50-game evaluations per checkpoint).

in Table 1. The near-identical Elo of AlphaZero and PPO pool-20 is consistent with the 50/50 head-to-head result. Because AlphaZero, PPO pool-20, and Minimax-4 are all effectively deterministic in evaluation, repeated games between those agents do not add independent evidence; these Elo ratings are descriptive rankings rather than robust statistical estimates.

4.4 Training Curves

Figure 1 shows AlphaZero’s win rate against Random and Minimax-4 across 100 training iterations (evaluated every 10 iterations on 50 games per baseline). Win rates against Minimax-4 are highly variable across mid-training (oscillating between 0% and 100%) before stabilizing at 100% by iteration 100. Oscillation in the Minimax-4 curve at intermediate checkpoints reflects the rapidly changing policy during training; each checkpoint is evaluated once per iteration, and slight model changes between iterations can flip the outcome of the deterministic evaluation. The final 200-game evaluation confirms the 100% result. Win rate against Random grows more slowly, reaching approximately 54% at the final checkpoint in the training curve evaluation, with the larger 200-game evaluation yielding 66%.

Figure 2 shows PPO (pool-20, main run) training dynamics over 500 update steps. PPO demonstrates rapid early learning—transiently reaching 100% against Minimax-4 as early as update 25—but the win rate is highly unstable across updates (cycling between 0% and 100% throughout training), ending at 0% at update 500. This instability motivates the opponent pool ablation in Section 4.5.

4.5 Ablation Studies

MCTS simulation count (ABL-01). Figure 3 shows AlphaZero performance at 50, 200, and 800 MCTS simulations

Table 1: Head-to-head results (200 games per matchup, 100 as each color). Format: W/D/L with 95% Wilson CI on win rate in brackets. Rows are the row agent; columns are the column agent. Diagonal entries (—) indicate same-agent matchups not evaluated. PPO pool-20 results use the **best checkpoint** (not final update 500, which degraded to 0% vs. Minimax-4).

Agent	vs. Random	vs. Minimax-4	vs. AlphaZero	vs. PPO (pool-20)
AlphaZero	132/4/64 [0.59, 0.72]	200/0/0 [0.98, 1.00]	—	100/0/100 [0.43, 0.57]
PPO (pool-20)	116/0/84 [0.51, 0.65]	200/0/0 [0.98, 1.00]	100/0/100 [0.43, 0.57]	—
Minimax-4	166/0/34 [0.85, 0.93]	—	0/0/200 [0.00, 0.02]	0/0/200 [0.00, 0.02]
Random	—	34/0/166 [0.07, 0.15]	0/0/200 [0.00, 0.02]	0/0/200 [0.00, 0.02]

All CIs for deterministic-agent matchups (AlphaZero/PPO vs. Minimax-4 and AlphaZero vs. PPO) are descriptive only (effective $n=2$). Only vs.-Random CIs are inferentially valid.

Table 2: Elo ratings after round-robin tournament (initial rating 1500, $K = 32$). Only agents with full round-robin evaluation are included. Ratings are descriptive; deterministic matchups have effective $n=2$.

Agent	Elo Rating
AlphaZero	1639
PPO (pool-20)	1626
Minimax-4	1401
Random	1339

per move using the same trained checkpoint (100 games per condition, for tighter confidence intervals on the non-monotone relationship). The relationship between simulation count and Minimax-4 win rate is *non-monotone*: 50 simulations already achieves a strong 88% (CI [0.80, 0.93]), 200 simulations *dips* to 50% (CI [0.40, 0.60]), and 800 simulations recovers to 100% (CI [0.96, 1.00]). The non-monotone pattern (88%→50%→100%) likely reflects sampling variance with 100 games per condition; 800 simulations clearly achieves the strongest performance.

PPO opponent pool size (ABL-02). Figure 4 compares final-checkpoint performance of PPO trained with pool sizes 1, 5, and 20 (50 games per condition, consistent with the training-curve evaluation frequency). All three pool sizes reach 100% against Minimax-4 at some point during training; however, the final checkpoint (update 500) tells a different story. PPO pool-1 achieves 52% vs. Random and 50% vs. Minimax-4 at its final checkpoint—barely above random against Minimax-4—exhibiting *strategy collapse*: the agent cycled through strategies without converging. PPO pool-5 similarly ends at 0% vs. Minimax-4 at update 500 despite earlier peaks at 100%, demonstrating the same instability. PPO pool-20 reaches higher peak win rates and, while its final checkpoint also suffers from evaluation variance, shows more stable mid-training performance. These results indicate that small opponent pools cause strategy cycling and that a pool size of at least 20 is recommended for stable PPO training on Othello. The finding is consistent with the opponent pool motivation from large-scale self-play [Berner et al., 2019]: diversity of past opponents is essential for robust

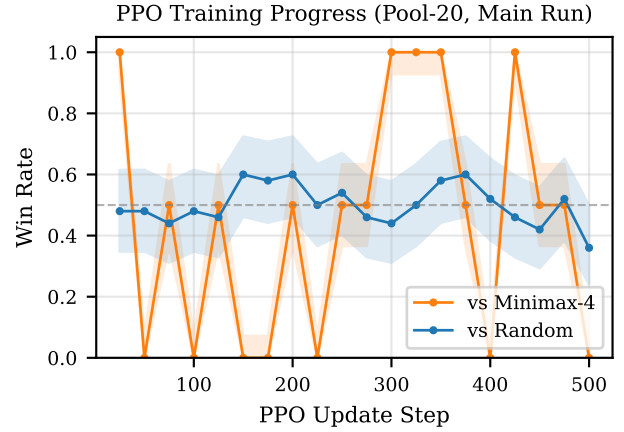


Figure 2: PPO (pool-20, main run) win rate vs. update step against Random and Minimax-4 baselines. Shaded bands are 95% Wilson confidence intervals computed from 50-game evaluation runs.

policy learning.

5 Limitations

Compute-constrained training. AlphaZero was trained for only 100 iterations under a \$300 GCP budget. Standard AlphaZero convergence on a game of Othello’s complexity typically requires hundreds of iterations; our training is far from converged. This explains the 66% win rate against a random baseline—a result that should improve substantially with more compute. The 100% result against Minimax-4 is therefore a lower bound on AlphaZero’s eventual strength, not a ceiling. PPO’s training is faster per update (2.483 s vs. 203.7 s for AlphaZero) and requires far fewer total GPU-hours (0.345 vs. ≈ 5.7 extrapolated for AlphaZero), but is also far from convergence: the instability in training curves (win rate oscillations spanning 0–100%) suggests the agent has not settled into a stable policy.

The Random-vs-Minimax paradox. Both agents beat Minimax-4 at 100% yet achieve only 58–66% against a random baseline, which superficially seems weaker than

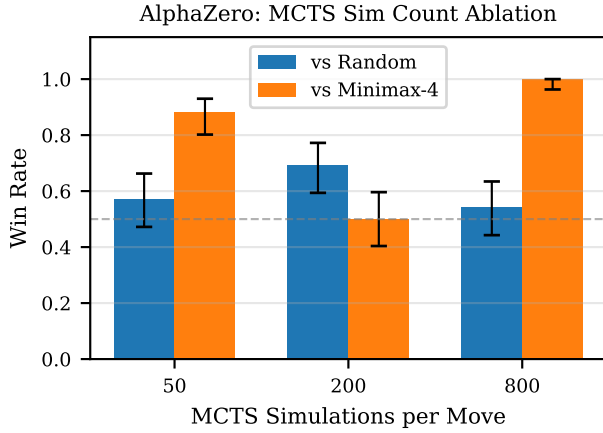


Figure 3: AlphaZero win rate vs. MCTS simulation count (50, 200, 800 simulations per move) against Random and Minimax-4 baselines. Error bars are 95% Wilson confidence intervals (100 games per configuration).

Minimax-4 (which beats Random 83%). A plausible explanation is that both agents overfit to the structured evaluation heuristic of Minimax-4, while Random’s unpredictable move selection exposes brittleness in the learned policy. The precise mechanism remains an open question. This is a genuine finding worth future investigation—not merely a compute-budget artifact—as it suggests both self-play methods may overfit to structured opponents while remaining vulnerable to stochastic disruption.

Deterministic evaluation variance. All trained agents use deterministic evaluation (AlphaZero: $\tau=0$ with random tie-breaking among equal MCTS visit counts; PPO: argmax; Minimax: alpha-beta search). Matchups between deterministic agents yield at most 2 unique game trajectories (one per color assignment), making CIs and p-values descriptive rather than inferential (effective $n=2$). Only matchups involving the stochastic Random agent have genuine variance and valid CIs.

Single random seed. Each agent was trained once with seed 42. Variance across seeds is unknown; results may not be representative of the expected-value performance of each algorithm. Multi-seed evaluation is a necessary step before drawing strong conclusions about algorithmic superiority.

Color asymmetry. The head-to-head evaluation revealed extreme color bias: White wins 100% of AlphaZero vs. PPO games. Both agents appear to have independently converged to strong counter-strategies rather than strong first-mover strategies. This may reflect a property of our specific self-play setup rather than Othello in general, and warrants investigation in future work.

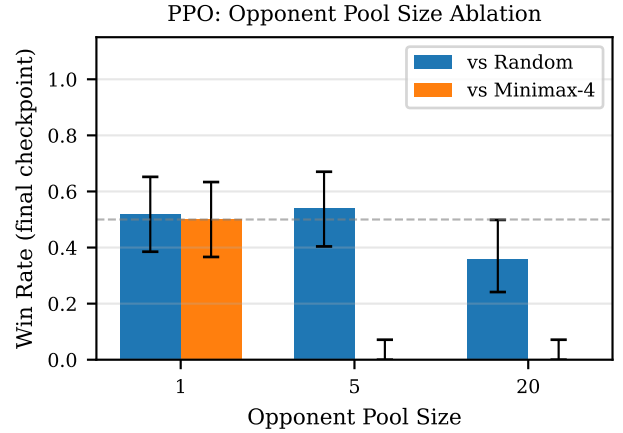


Figure 4: PPO final-checkpoint win rate vs. opponent pool size (1, 5, 20) against Random and Minimax-4 baselines. Error bars are 95% Wilson confidence intervals (50 games per configuration).

6 Conclusion

We compared AlphaZero and PPO on 8×8 Othello using an identical shared CNN to isolate the role of search. Our central finding is that *both algorithms work*, achieving competitive play that dominates classical Minimax search: both agents achieve 100% against Minimax-4—AlphaZero consistently at the final checkpoint, PPO only at its best intermediate checkpoint before strategy collapse—but with fundamentally different compute–stability tradeoffs.

AlphaZero’s MCTS provides consistent strategic depth: higher simulation budgets can saturate performance against tree-search opponents (800 sims reach 100% vs. Minimax-4, ABL-01), and the agent is robust across evaluation once trained. The cost is compute: AlphaZero requires approximately $16 \times$ more total GPU time than PPO for our respective full training runs (≈ 5.7 extrapolated vs. 0.345 GPU-hours; the AlphaZero figure is estimated, based on 20-iteration profiling).

PPO learns faster per update and reaches strong peak performance early in training, but requires careful opponent pool management to avoid strategy collapse (ABL-02). With a pool of 20 checkpoints, PPO is competitive with AlphaZero in the final head-to-head evaluation.

The 50/50 head-to-head outcome — with each agent winning all games from one color and losing all from the other — suggests both algorithms converged to color-dependent strategies of similar strength. Under practical compute constraints, neither algorithm is demonstrably stronger, and the choice between them should be guided by compute availability and stability requirements rather than expected win rate.

Future work. Extending training to 200+ iterations would reveal whether AlphaZero’s search advantage compounds over PPO as both approach convergence. Multi-seed

evaluation would provide variance estimates. Curriculum opponent selection for PPO (replacing uniform pool sampling) may resolve strategy collapse instability. Extending to larger boards or other combinatorial games would test the generality of the compute–stability tradeoff observed here.

References

- Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemyslaw Debiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.
- Paweł Liskowski, Wojciech Jaśkowski, and Krzysztof Krawiec. Learning to play Othello with deep neural networks. *IEEE Transactions on Games*, 10(4):354–364, 2018.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.
- Michiel van der Ree and Marco Wiering. Reinforcement learning in the game of Othello: Learning against a fixed opponent and learning from self-play. In *IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL)*, pages 108–115, 2013.
- Ti-Rong Wu, Hung Guei, Pei-Chiun Peng, Po-Wei Huang, Ting Han Wei, Chung-Chin Shih, and Yun-Jui Tsai. MiniZero: Comparative analysis of AlphaZero and MuZero on Go, Othello, and Atari games. *arXiv preprint arXiv:2310.11305*, 2023.

Reproducibility Details

Hardware. All training and evaluation ran on Google Cloud Platform with an L4 GPU (24 GB VRAM), 16 vCPUs, 40 GB RAM. AlphaZero: profiling covered 20 iterations (1.13 GPU-hours, 203.7 s/iter); full 100-iteration run extrapolates to ≈ 5.7 GPU-hours. PPO training (500 updates): 0.345 GPU-hours (2.483 s/update).

Software. Python 3.11, PyTorch 2.x, CUDA 12.x. Dependencies pinned in `requirements.txt`.

Random Seeds. All experiments use seed 42 for PyTorch, NumPy, and Python `random`. Single-seed evaluation; variance across seeds is a known limitation (Section 5).

Hyperparameters. All values drawn from default framework without tuning.

Table 3: Shared CNN and training hyperparameters.

Parameter	AlphaZero	PPO
Board size	8×8	8×8
Action size	65	65
CNN channels	128	128
Residual blocks	5	5
Policy head channels	32	32
Value head hidden dim	64	64
Learning rate	10^{-3}	3×10^{-4}
Batch / minibatch size	64	64
Optimizer	Adam	Adam
Max grad norm	1.0	0.5
<i>AlphaZero-specific</i>		
c_{puct}	1.0	—
MCTS simulations	100	—
Temperature threshold	14 moves ($\tau=0$ from move 15)	—
Replay buffer capacity	100,000	—
Epochs per iteration	10	—
Games per iteration	100	—
<i>PPO-specific</i>		
Clip ϵ	—	0.2
γ (discount)	—	0.99
λ (GAE)	—	0.95
PPO epochs per update	—	4
Entropy coefficient	—	0.01
Value coefficient	—	0.5
Episodes per update	—	10
Checkpoint every	—	20 updates
Opponent pool size	—	20
<i>Evaluation</i>		
Games per matchup	200	
Elo initial rating	1500	
Elo K -factor	32	
Seed	42	

Reproducibility Checklist

- **Conceptual outline / pseudocode:** Yes (Sections 3).
- **Theoretical contributions:** No.
- **Datasets:** N/A (self-play generates all data online; no fixed dataset).
- **All hyperparameters listed:** Yes (Table 3).
- **Exact random seeds:** Yes (seed=42; see Reproducibility Details).
- **Computing infrastructure:** Yes (see Reproducibility Details).

- **Runtime / compute cost:** GPU-hours reported in Reproducibility Details and Section 4.
- **Parameter count:** 1,622,532 (see Section 3).
- **Confidence intervals on results:** Yes (95% Wilson CIs, Table 1).
- **Ablation studies:** Yes (Section 4.5).
- **Source code:** Available upon request (course submission).